

Grundarkitektur för en flöde

Max/MSP & Java

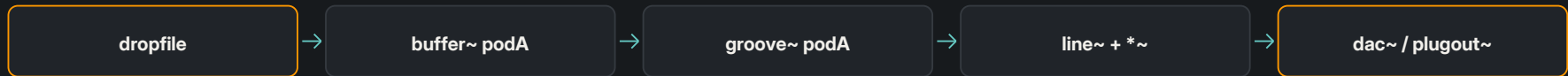
Teknisk utvecklarhandbok för POD-arkitektur, DSP, generering, state och routing

TEKNISK ANMÄRKNING Nuvarande generator använder JavaScript for Max via js, inte Java/mxj.

Nuläge och teknisk sanning

Vad som finns, vad som är partiellt och vad som fortfarande är målarkitektur.

NUVARANDE ONE-POD-KANDIDAT



flode_podA_MVP_v1.maxpat har sample-load, buffer, waveform, groove-playback, klickfri gain och en JavaScript-baserad generator. Generatoren påverkar för närvarande endast playback speed.

IMPLEMENTERAT ELLER STRUKTURELLT OBSERVERAT

- buffer~ + waveform~ + groove~
- live.slider för Volume och Gen-parametrar
- js flode_gen.js med sex modes
- Separat sex-POD-experiment med gemensam step-clock

INTE FÄRDIGKOPPLAT

- Pan, Mute, Solo och Loop i current POD
- Position- och velocity-intent till slice/playback
- Host-sync, state restore och choke groups
- Pod FX, master FX och slutlig sex-POD-integration

RUNTIME-STATUS

Arkitekturen är strukturellt läsbar i .maxpat och JavaScript, men runtime-gates i Max 9 och Ableton Live 12 är fortfarande avgörande. En designbild eller giltig JSON-patch är inte bevis på stabil DSP, korrekt state eller säker host-routing.

En kanonisk POD-abstraktion

En återanvändbar intern struktur instansieras A-F utan kopierad, divergerande DSP.

PRESENTATION

bpatcher / live.* / jsui

Ritar och tar emot gester. Äger aldrig audio-state.

STATE

pattr / autopattr / pattrstorage

Lagrade grundvärden, locks, preset och återställning.

EVENT

js flode_gen.js / transport / gate

Beslutar när, var och hur nästa event ska spelas.

DSP

buffer~ / groove~ / line~ / *~

Laddar, spelar och formar audio med klickfria övergångar.

ROUTING

send~ / matrix~ / plugout~

Kopplar POD till FX, master och Live utan global kollision.

Rekommenderad instansiering: parent-device skickar ett device-scope som #1 och POD-id som #2. Buffert kan heta #1.pod.#2.buffer och delad clock #1.flode.step. Det förhindrar kollision när samma .amxd dupliceras i Live.

Box 1 – Waveform och header

Sample-minne, loopgränser, playhead, slices och PODens fasta performance-state.



1.1 Load

dropfile eller opendialog skickar replace till buffer~.

1.2 Waveform

waveform~ refererar samma buffertnamn och visar sample.

1.3 Start/End

selection-utgångarna ger loopgränser i millisekunder.

1.4 Playhead

groove~ sync/phase samplas för en visuell positionslinje.

1.5 Zoom

display-start och display-length skickas till waveform~.

1.6 Slices

analys skapar markerlista; jsui ritar overlay ovanpå waveform~.

1.7 Loop/Once

loop, startloop, start och stop styr groove~.

1.8 Sync/Free

host-transport eller intern scope:ad clock väljs explicit.

1.9 Record

record~ skriver till buffer; srecord~ skriver till disk.

1.10 Header-state

Mute, Solo, Rec och status skickas till state/mixer, inte direkt till UI-logik.

TRANSIENTANALYS

Package-fri MVP: manuella slices och eventuell bonk~-analys. Robust offline-analys: FluCoMa TransientSlice som uttryckligen godkänd extern dependency. Markörer lagras i coll, dict eller JavaScript-array.

Box 2 - Playback

Playback rate, riktning, pitch, transienttäthet och uppspelningstyp.



- 2.1 Speed**
live.slider skalar rate; line~ eller sig~ ger stabil signalstyrning.
- 2.2 Direction**
positiv rate spelar framåt och negativ rate spelar bakåt.
- 2.3 Pitch**
separat pitch kräver timestretch/pitch-processering, inte bara ändrad rate.
- 2.4 Transient Density**
väljer hur stor del av markerlistan som får triggas.

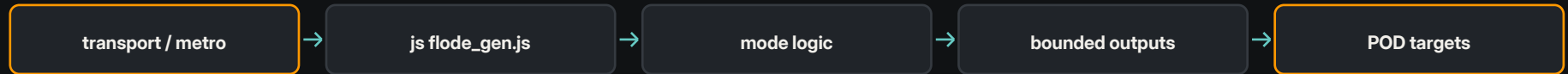
- 2.5 Loop**
groove~ återupprepar intervallet Start-End.
- 2.6 Once**
ett accepterat event spelar området en gång och stoppar.
- 2.7 Orange värde**
användarens sparade grundvärde och automationsmål.
- 2.8 Turkos markör**
aktuellt genererat/modulerat värde utan att skriva över basen.

CPU-BESLUT

Rate-baserad playback är billig och ändrar pitch tillsammans med hastighet. Oberoende Pitch och Speed kräver groove~ timestretch-läge eller stretch~ och måste budgeteras per sex samtidiga PODs.

Box 3 - Generative modes

Nuvarande JavaScript-motor producerar speed, position intent och velocity intent.



PURE

Oberoende uniform slump utan viktning eller historik.

JUNG

Slump med återupprepning, pauser och sällsynta extremvärden.

WEIGHTED

Värden klustras kring musikaliskt användbara regioner.

WALK

Begränsad förflyttning från föregående state.

MEMORY

Rullande historik återanvänds som självskapade mönster.

CHAOS

Deterministisk nonlinear feedback inom säkra gränser.

JavaScript for Max: objektet `js` laddar `max/current/js/flode_gen.js`. Java/`mxj` används inte i nuvarande arkitektur och tillför ingen nödvändig MVP-funktion.

Generative controls och GEN-vyn

Makron, sannolikhet, locks, memory och begränsad positionsgenerering.

3.1

Random/Jung

skalar hur långt generatoren får röra sig från grundvärdet.

3.2

Jitter

liten eventbaserad avvikelse efter den valda mode-logiken.

3.3

Probability

random 100 jämförs med threshold och öppnar en gate.

3.4

Lock Off

genererade resultat får uppdatera målet normalt.

3.5

Lock Hold

senaste värdet hålls med JS-state, gate eller zl reg.

3.6

Lock Latch

värdet hålls tills uttrycklig upplåsning eller ny capture.

3.7

Repeat/Pause

nya kontroller ersätter hårdkodade sannolikheter i JUNG.

3.8

Memory Depth

parameteriserar storleken på generators memory-array.

3.9

Position Range

skalar 0-1 intent till ett säkert sample- eller slice-område.

3.10

Velocity Intent

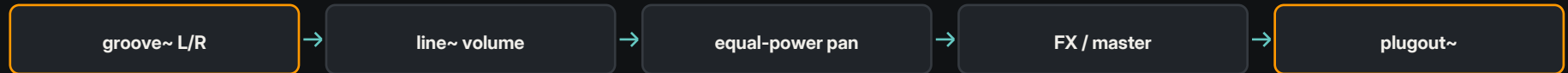
mappas till event-gain via klickfri line~~ramp.

NUVARANDE KOPPLINGSLUCKA

flode_gen.js lämnar redan speed, position och velocity. I current POD går endast speed vidare till groove~. Position och velocity är avsiktligt framtida intent tills slice-trigger och gain-routing är implementerade och runtime-testade.

Box 4 - Mix & Route

Klickfri nivå, equal-power pan, meters, FX send, choke och host-output.



4.1 Volume

pack 0. 20 till line~ och separata *~ för klickfri gain.

4.2 Pan

sqrt-baserad equal-power-formel ger separata vänster/höger gains.

4.3 Meter

meter~ eller live.meter~ efter POD-mix och före master.

4.4 Mute

rampar POD-gain till noll men låter clock och memory fortsätta.

4.5 Solo

central arbiter öppnar endast gains för solo-aktiva PODs.

4.6 FX Send

scope:ad send~/receive~ eller matrix~ för stabil routing.

4.7 Choke

control-event stoppar eller tonar ut föregående POD i gruppen.

4.8 Output

plugout~ används i .amxd; dac~/ezdac~ endast i utvecklingspatch.

SÄKERHETSREGEL

Current POD skickar summerad signal till både dac~ och plugout~ och har ofullständig Pan/Mute/Solo-routing. Canonical POD måste välja en miljöanpassad output-väg och undvika dubbla eller globala audio-bussar.

Box 5 - Sample, Gen, Slice och FX

Flikarna byter presentation. Audio engine och state fortsätter utan avbrott.

SAMPLE

`buffer~ / info~ / dict`

Källfil, längd, kanaler, relink, replace, normalize och grundområde.

GEN

`js / jsui / mgraphics`

Tendency-graf, memory-kurva, position range och generativ aktivitet.

SLICE

`analysis / coll / waveform~`

Skapa, flytta, ta bort, vikta och trigga markerpositioner.

FX

`svf~ / biquad~ / tapin~`

Filter, delay, saturation, EQ, bypass, mix och ordningsstate.

UI-implementation: live.tab väljer synlig bpatcher/panel. DSP-abstraktionen ligger kvar laddad; ett flikbyte får aldrig återladda buffer, stoppa audio eller återställa generator-state.

09 State, automation och parameterkontrakt

Varje POD behöver stabila namn, dokumenterade ranges och definierad recall-ordning.

PARAMETER PATH EXAMPLES

podA.speed

podA.pitch

podA.loop.start

podA.gen.mode

podA.gen.amount

podA.mix.volume

podA.fx.filter.cutoff

autopattr

Registrerar namngivna UI-objekt och deras lagrade värden.

pattr

Binder explicit state, listor eller aliases till ett stabilt namn.

pattrstorage

Sparar presets, återkallar state och hanterar recall-prioritet.

live.* parameters

Exponerar range, units, automation, mapping och undo i Live.

dict / JS state

Lagrar markerlistor, memory-bank och strukturerad metadata.

locks

Sparas separat från basvärde och aktuellt genererat resultat.

Sex PODs A-F och global state

En POD-abstraktion, sex instanser och ett explicit masterlager.

POD A

#1.pod.A.buffer

same abstraction / unique state

POD B

#1.pod.B.buffer

same abstraction / unique state

POD C

#1.pod.C.buffer

same abstraction / unique state

POD D

#1.pod.D.buffer

same abstraction / unique state

POD E

#1.pod.E.buffer

same abstraction / unique state

POD F

#1.pod.F.buffer

same abstraction / unique state

MASTER OWNS

Host/MIDI/internal clock, BPM, transport, quantisation, swing, solo arbitration, global variation, master FX, limiter, recorder, output och Panic.

Undvik globala namn som flode.step utan device-scope. Annars påverkar två instanser av .amxd varandra i samma Live Set.

Libraries och dependency-policy

MVP ska vara portabel, spårbar och fri från onödiga externa packages.

Max 9	REQUIRED	Patch, UI, scheduling och project runtime.
MSP	REQUIRED	buffer~, groove~, signalflöde, gain och routing.
Max for Live	REQUIRED	plugout~, host transport, automation och .amxd.
JavaScript for Max	REQUIRED	flode_gen.js och avancerad control/UI-logik.
jsui / mgraphics	BUILT-IN	Slice overlays, curves och modulation feedback.
Gen~	OPTIONAL	Saturation, special-DSP och optimerade interna processer.
FluCoMa	OPTIONAL EXTERNAL	Robust transient slicing och analys.
Jitter	NOT REQUIRED	Behövs inte för POD-MVP.
RNBO	LATER	Relevant först vid export eller standalone-strategi.
Figma / p5.js / React	REFERENCE ONLY	Visuell prototyp, aldrig DSP-bevis.

Rekommenderad MVP-bas: Max + MSP + Max for Live + JavaScript. Externa packages införs först efter dokumenterat godkännande och test på målmaskinen.

Rekommenderad byggordning

Varje steg avslutas med ett runtime-gate innan nästa lager byggs ovanpå.

1

Canonical POD A

Sample load, waveform, loop, playhead, safe stereo output.

2

Mixer och state

Volume, pan, mute, solo, Panic, stable names och restore.

3

Slice engine

Markeranalys, density, selection och deterministic triggering.

4

Generative connection

Position, velocity, locks, memory depth och ranges.

5

Six POD integration

A-F, clock scope, choke, meters och Focus View.

6

FX och master

Pod FX, reorderable master, limiter, recorder och gain staging.

7

Max for Live package

Host-sync, automation, save/reload, duplicate track och .amxd.

Validation och källor

Definitionen av färdig funktion kräver target-runtime, inte bara strukturell kontroll.

VARJE .MAXPAT-ÄNDRING

- Giltig JSON och alla dependency paths finns.
- Ingen dubbel output, feedback eller global namnkonflikt.
- Max 9: Presentation Mode och ren Max Console.
- Audio on/off, load, loop, speed, gain, pan och Panic.
- 30 min stabil POD utan spikes eller state corruption.
- Ableton Live 12: transport, tempo, automation och restore.

PRIMÄRA TEKNISKA KÄLLOR

Cycling '74 waveform~

Cycling '74 groove~

Cycling '74 buffer~

Cycling '74 pattrstorage

Cycling '74 live.slider

FluCoMa reference

REPOSITORY SOURCE OF TRUTH

PROJECT_CONTEXT.md

ROADMAP.md

flode_gen.js

Om Max 9 eller Live 12 inte är tillgängliga ska resultatet alltid märkas: structurally validated, runtime unverified.